

Assignment: HW/SW Co-Design of a Driver Assistance System

by

Md Mostafizur Rahman

Introduction

A modern driver assistance system is used to detect objects such as vehicles, pedestrians, cyclists, and obstacles in real time. After detecting an object, the system must respond quickly, for example by warning the driver or activating the braking system.

The system consists mainly of a camera, a processing unit, and actuators such as the braking system. Since this is a safety-critical system, it must have low latency, high reliability, low energy consumption, and enough flexibility for future software updates.

a) System Partitioning

System partitioning means deciding which system parts should be implemented in hardware and which parts should be implemented in software.

Hardware Components

Camera Sensor and Image Acquisition

The camera sensor should be connected using a high-speed hardware interface such as MIPI CSI-2. The camera continuously captures images of the road environment, so the image data must be transferred quickly and reliably.

This task is suitable for hardware because image acquisition requires real-time performance and predictable data transfer. A CPU alone should not handle this raw image transfer because it would increase delay and energy consumption.

Image Pre-processing

Image pre-processing should be implemented using an FPGA, Image Signal Processor, or DSP unit.

Pre-processing task	Purpose
Noise filtering	Removes unwanted noise from the image
Scaling	Reduces image size for faster processing
Color conversion	Converts image format, for example RGB to grayscale
Edge enhancement	Improves object boundaries

An FPGA is suitable because these operations are repetitive and can be performed in parallel. This gives low latency and good energy efficiency.

Object Detection Acceleration

Object detection should be accelerated using a GPU, NPU, or dedicated AI accelerator.

Modern object detection algorithms such as YOLO, SSD, or MobileNet-SSD require many mathematical operations, especially convolution and matrix multiplication. A normal CPU may not be fast enough for real-time object detection.

A GPU is useful because it can perform many operations in parallel. An NPU is even more efficient for neural network inference. An FPGA can also be used when predictable timing and low power consumption are important.

Technology	Example
GPU-based embedded system	NVIDIA Jetson
Automotive SoC	NXP S32V, TI TDA4VM
FPGA platform	AMD/Xilinx Zynq UltraScale+
Safety microcontroller	Infineon AURIX, NXP S32K

Actuator Control

The low-level actuator control, especially braking, should be handled by a real-time automotive microcontroller such as Infineon AURIX or NXP S32K.

Braking is safety-critical, so it must be deterministic. This means the command must be executed within a fixed time. The microcontroller can also check whether the braking command is valid before sending it to the actuator.

Software Components

Decision-Making Logic

The decision-making logic should run on an embedded CPU or automotive SoC. It decides what action the system should take after object detection.

Situation	Decision
Pedestrian far away	Continue monitoring
Vehicle ahead slowing down	Warn the driver
Obstacle very close	Activate braking

This part should be software-based because the decision rules may need to be updated or improved later.

Object Tracking

Object tracking should be implemented in software. Object detection identifies objects in one frame, while tracking follows the same object over several frames. This helps the system understand whether an object is moving closer, moving away, or crossing the road.

Sensor Fusion

If the system uses other sensors such as radar, lidar, or ultrasonic sensors, the data should be combined using software. For example, the camera can classify an object as a pedestrian, while radar can measure the distance and speed of that object. Sensor fusion improves safety and reduces false detection.

Diagnostics and Monitoring

Diagnostics and monitoring should also be implemented in software. The system should continuously check whether the camera, processor, memory, and communication interfaces are working correctly. Software can also handle logging, self-tests, communication, and software updates.

Final Decision

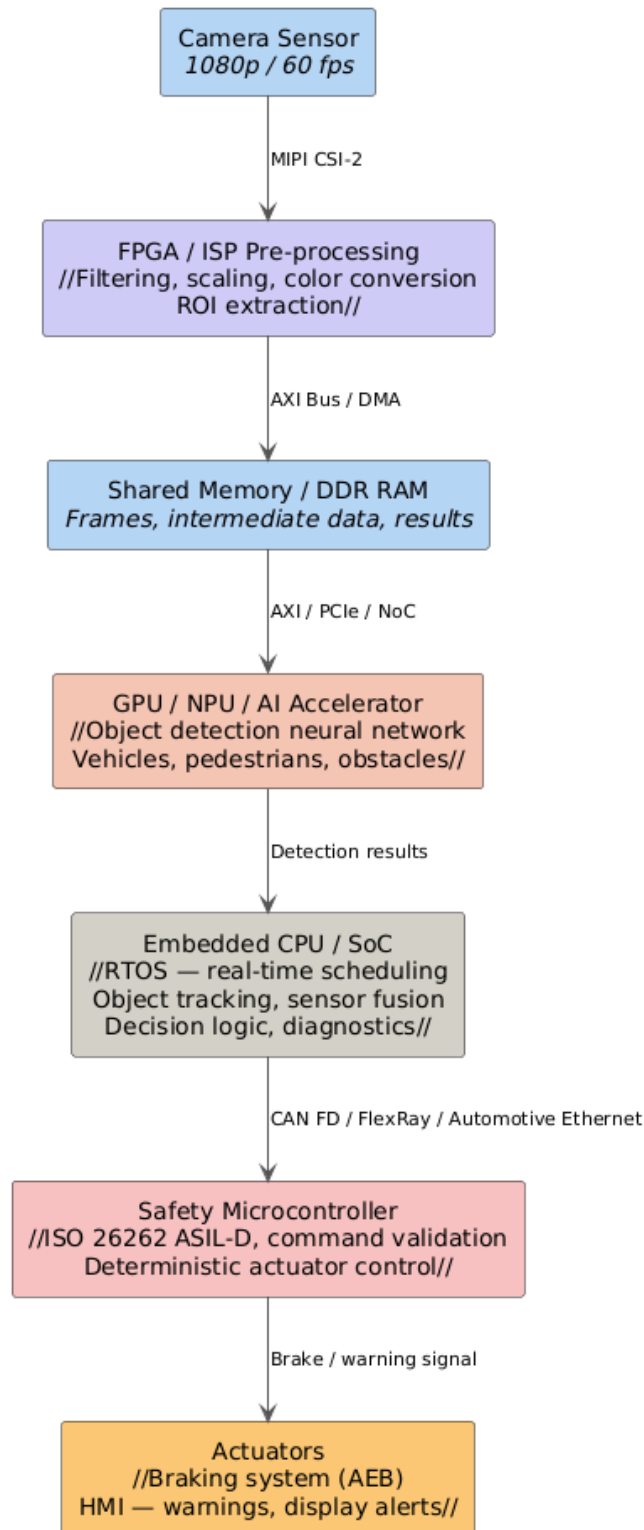
System function	Hardware/Software	Examples	Reason
Camera capture	Hardware	MIPI CSI-2	Fast and reliable image transfer
Image pre-processing	Hardware	FPGA, ISP, DSP	Parallel processing with low latency
Object detection	Hardware accelerator	GPU, NPU, FPGA	Fast neural network inference
Decision-making	Software	Embedded CPU with RTOS	Flexible and updateable
Object tracking	Software	Embedded SoC	Algorithm can be changed
Sensor fusion	Software	CPU / SoC	Flexible combination of sensor data
Braking control	Hardware / MCU	Infineon AURIX, NXP S32K	Deterministic safety-critical control
Diagnostics	Software	RTOS task	Easy monitoring and updates

Hardware is chosen for tasks where need high speed, low latency, and energy efficiency. Software is chosen for tasks that require flexibility and future updates.

b) Architecture Design

The driver assistance system can be designed using a camera, pre-processing hardware, shared memory, AI accelerator, embedded processor, safety microcontroller, and actuators.

Block Diagram



Components

I. Camera Sensor

The camera captures images of the road. It sends video data to the processing system using MIPI CSI-2, which is suitable for high-speed camera data transfer.

II. Pre-processing Unit

The pre-processing unit prepares the image for object detection. It can be implemented using an FPGA, ISP, or DSP. This block performs filtering, scaling, and color conversion.

III. Shared Memory

The shared memory, usually DDR RAM, stores camera frames, intermediate data, and detection results. It allows the CPU and accelerator to access the required data.

IV. AI Accelerator

The AI accelerator performs object detection. It may be a GPU, NPU, or FPGA-based accelerator. It runs the neural network model and detects vehicles, pedestrians, and other objects.

V. Embedded CPU / SoC

The embedded processor runs the main software. It can run an RTOS to guarantee real-time task scheduling. It performs object tracking, sensor fusion, decision-making, diagnostics, and communication.

VI. Safety Microcontroller

The safety microcontroller receives commands from the CPU and checks whether they are safe. It controls the braking system or warning system. It is used because actuator control must be reliable and deterministic.

VII. Actuators

The actuators perform the final action. Examples include braking, warning lights, warning sounds, or display messages.

Data Flow

First, the camera captures road images and sends them to the pre-processing unit through MIPI CSI-2.

Next, the pre-processing unit improves the image and stores it in DDR memory using DMA. DMA is useful because it transfers data without heavily loading the CPU.

Then, the GPU, NPU, or FPGA accelerator reads the image data from memory and performs object detection.

After that, the detection results are sent to the embedded CPU. The CPU tracks objects, calculates risk, and decides whether action is needed.

Finally, if a dangerous situation is detected, the CPU sends a command through CAN bus, FlexRay, or Automotive Ethernet to the safety microcontroller. The microcontroller validates the command and activates the actuator.

Interfaces

Interface	Purpose
MIPI CSI-2	Camera to processing unit
AXI Bus	Internal SoC communication
DMA	Fast memory transfer without high CPU load
PCIe	Connection to high-performance accelerator
CAN Bus	Reliable vehicle communication
FlexRay	Deterministic automotive communication
Automotive Ethernet	High-speed vehicle network
SPI / I2C	Sensor configuration
PWM / actuator interface	Direct actuator control

c) Performance Analysis

The system must detect objects and react in real time. If the response is too slow, the vehicle may not brake early enough.

Latency Requirement

A reasonable target for the total system latency is around 50 ms to 100 ms. This includes image capture, image processing, object detection, decision-making, communication, and actuator command generation.

At a speed of 100 km/h, a vehicle travels about 27.8 meters per second. Therefore, in 100 ms, the vehicle travels about 2.78 meters. This shows why low latency is very important.

Estimated Latency

Processing stage	Estimated latency
Camera capture and transfer	10-20 ms
Image pre-processing	2-5 ms
Object detection using GPU/NPU	10-30 ms
Tracking and decision-making	5-10 ms
Communication to actuator controller	1-5 ms
Actuator command processing	5-20 ms
Total estimated latency	40-90 ms

This estimated latency can satisfy real-time requirements if the hardware and software are optimized properly.

Potential Bottlenecks

Image Data Transfer

High-resolution cameras generate a large amount of data. If the bus or memory bandwidth is too low, the system may become slow.

Neural Network Computation

Object detection requires many calculations. Running the neural network only on a CPU may not meet real-time requirements. This is why a GPU, NPU, or FPGA accelerator is needed.

Memory Bandwidth

The camera, CPU, and AI accelerator all need access to memory. If the memory bandwidth is limited, the accelerator may wait for data, increasing latency.

Communication Delay

The braking command must reach the actuator controller quickly. Communication should be deterministic, especially for safety-critical functions.

Thermal Limitation

GPUs and high-performance processors can generate heat. If the system becomes too hot, performance may be reduced automatically. This can affect real-time behavior.

Optimization Strategies

Use Hardware Accelerators

A GPU, NPU, or FPGA should be used for object detection. This reduces computation time compared to a normal CPU.

Use Parallel Processing

The system should process different frames at the same time. For example: Frame $n+1$ is captured by the camera, Frame n is processed by the AI accelerator, and Frame $n-1$ is used by the CPU for decision-making.

Use DMA

Direct Memory Access should be used for image transfers. DMA reduces CPU load and allows faster data movement between camera, memory, and accelerator.

Optimize the Neural Network

Method	Benefit
Quantization	Reduces model size and computation
Pruning	Removes unnecessary parameters
YOLO-Tiny / MobileNet-SSD	Lightweight models for embedded systems
Batch size 1	Reduces real-time latency

Use RTOS Scheduling

An RTOS should be used so that safety-critical tasks get higher priority. This helps ensure that important tasks finish within their deadlines.

d) Verification and Safety

Because the system can control braking, it must be tested carefully and designed according to functional safety principles.

Verification Methods

Simulation

Simulation is used to test the system before real hardware testing. Different driving scenarios can be simulated.

Scenario	Purpose
Pedestrian crossing	Test object detection and braking
Vehicle suddenly braking	Test reaction time
Night driving	Test camera reliability
Rain, fog, or snow	Test difficult weather conditions
Sensor failure	Test fault handling

Software-in-the-Loop Testing

In Software-in-the-Loop testing, the software is tested on a computer using simulated sensor data. This helps check the decision-making logic, object tracking, and sensor fusion before using real hardware.

Hardware-in-the-Loop Testing

In Hardware-in-the-Loop testing, the real embedded hardware is connected to a simulator. The simulator provides sensor inputs, and the hardware produces actuator commands. This is important because it tests the real-time behavior of the actual hardware without needing a real vehicle.

Vehicle Testing

After simulation and HIL testing, the system should be tested in a real vehicle under controlled conditions, such as a closed test track. Tests should include emergency braking, pedestrian detection, different speeds, and different lighting conditions.

Formal Verification

Formal verification can be used for safety-critical software parts, such as braking decision logic. It helps prove that the system does not enter unsafe states under defined conditions.

Safety Requirements

The system should follow automotive functional safety standards such as ISO 26262.

Safety requirement	Explanation
Fail-safe behavior	System enters a safe state if a fault occurs
Redundancy	Critical components can be duplicated
Watchdog timer	Detects software or processor failure
Error detection	Uses CRC, parity, and ECC memory
Real-time response	System must react before the deadline
Safe actuator control	Braking command must be validated
Diagnostics	System continuously monitors itself

For braking-related functions, the safety level may be high, for example ASIL C or ASIL D, depending on the hazard analysis.

Handling Faults During Operation

Camera Fault

If the camera is blocked, disconnected, or damaged, the system should detect this using image quality checks or camera status monitoring. The driver should be warned, and automatic braking may be disabled if reliable detection is not possible.

Processor Fault

A watchdog timer should monitor the CPU and AI accelerator. If the processor stops responding, the safety microcontroller should put the system into a safe state.

Memory Fault

The system should use ECC memory to detect and correct memory errors. Important data should also be protected using checksums.

Communication Fault

Messages between the processor and actuator controller should include error detection such as CRC. If a message is missing, delayed, or corrupted, the actuator controller should reject it.

False Detection

The system may sometimes detect an object incorrectly. To reduce false positives and false negatives, the system should use confidence thresholds, object tracking, and sensor fusion. For example, radar can confirm the distance and speed of an object detected by the camera.